HBnB : Documentation Technique

1. Introduction

1.1 Objectif du document

Ce document présente la conception architecturale de l'application **HBnB Évolution** dans le cadre de la Partie 1 du projet.

Son objectif est de **définir une base technique claire et structurée** avant la phase d'implémentation.

Il formalise :

- L'architecture globale du système
- La conception de la logique métier
- Les interactions entre les différentes couches
- Les règles métier à respecter

Ce document sert de **référence technique** pour les prochaines phases du projet.

1.2 Présentation du projet

HBnB Evolution est une application inspirée des plateformes de réservation de logements. Elle permet :

- La gestion des utilisateurs
- La gestion des logements
- La gestion des avis
- La gestion des équipements

Les principales entités du système sont :

- User
- Place
- Review
- Amenity

Chaque entité possède :

- Un identifiant unique (UUID)

- Un champ created_at
- Un champ updated_at

Ces éléments garantissent la **traçabilité et l'intégrité des données**.

2. Architecture de Haut Niveau

2.1 Objectif du diagramme de package

Le diagramme de package illustre :

- L'architecture en trois couches
- Les dépendances entre les couches
- L'utilisation du pattern Façade
- L'absence de dépendances circulaires

2.2 Organisation en trois couches

L'architecture de HBnB repose sur une organisation en **trois couches distinctes**

: • PresentationLayer

- BusinessLogicLayer
- PersistenceLayer

Cette séparation permet d'assurer un **découplage clair** entre les responsabilités.

Explication des flèches :

➜ PresentationLayer → BusinessLogicLayer
Cette flèche signifie que :

- L'API dépend de la logique métier
- L'API appelle uniquement la façade
- L'API ne contient aucune règle métier

La direction de la flèche montre que la couche Présentation connaît la couche Business,

mais **pas l'inverse**.

➜ BusinessLogicLayer → PersistenceLayer
Cette flèche indique que :

- La logique métier utilise les repositories
- Elle délègue la persistance des données
- Elle ne connaît pas la base de données directement

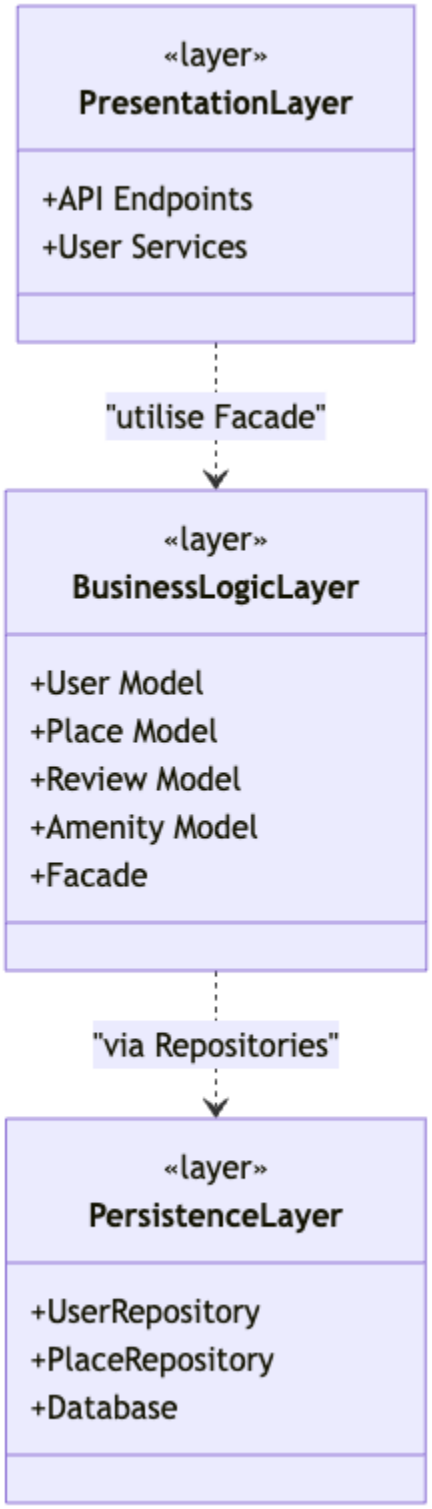
Absence de flèche inverse

La couche Persistance :

- Ne connaît pas l'API
- Ne connaît pas les règles métier

Cela garantit l'**absence de dépendance circulaire**.

DIAGRAMME DE PACKAGES UML


«layer»
PresentationLayer
+API Endpoints
+User Services
"utilise Facade"
«layer»
BusinessLogicLayer
+User Model
+Place Model
+Review Model
+Amenity Model
+Facade
"via Repositories"
«layer»
PersistenceLayer
+UserRepository
+PlaceRepository
+Database

2.3 Architecture en couches - Détails

L'application suit une architecture en **trois couches** :

2.3.1 Couche Présentation (Presentation Layer)

Responsabilités :

- Exposer l'API REST
- Recevoir les requêtes HTTP
- Valider les données d'entrée
- Retourner les réponses HTTP
- Appeler la façade

■ Cette couche **ne contient aucune logique métier**

Elle agit uniquement comme **point d'entrée** du système

2.3.2 Couche Logique Métier (Business Logic Layer)

C'est le **cœur du système**.

Elle contient :

- Les entités : User, Place, Review, Amenity
- Les règles métier
- Les validations fonctionnelles
- La classe Facade

Cette couche garantit la **cohérence du système**.

Exemples de règles métier :

- Le prix d'un logement doit être positif
- La note d'un avis doit être comprise entre 1 et 5
- Un utilisateur ne peut laisser qu'un seul avis par logement
- Un logement doit obligatoirement avoir un propriétaire

La logique métier est **indépendante** des couches Présentation et Persistance, ce qui garantit une architecture **modulaire et évolutive**.

2.3.3 Couche Persistance (Persistence Layer)

Responsabilités :

- Gérer l'accès à la base de données
- Implémenter les opérations CRUD
- Isoler la technologie de stockage

Elle contient :

- UserRepository
- PlaceRepository
- ReviewRepository
- AmenityRepository

Cette séparation permet de **modifier la base de données** sans impacter la logique métier.

2.4 Justification du pattern Façade

La classe **Façade** centralise toutes les opérations métier.

Au lieu de laisser l'API manipuler directement les entités, **toutes les requêtes passent par la façade**.

Avantages :

- Réduction du couplage
- Simplification de l'interface exposée
- Centralisation des traitements
- Meilleure maintenabilité

La façade agit comme un **point d'entrée unique** vers la logique métier.

3. Couche Logique Métier – Diagramme de Classes

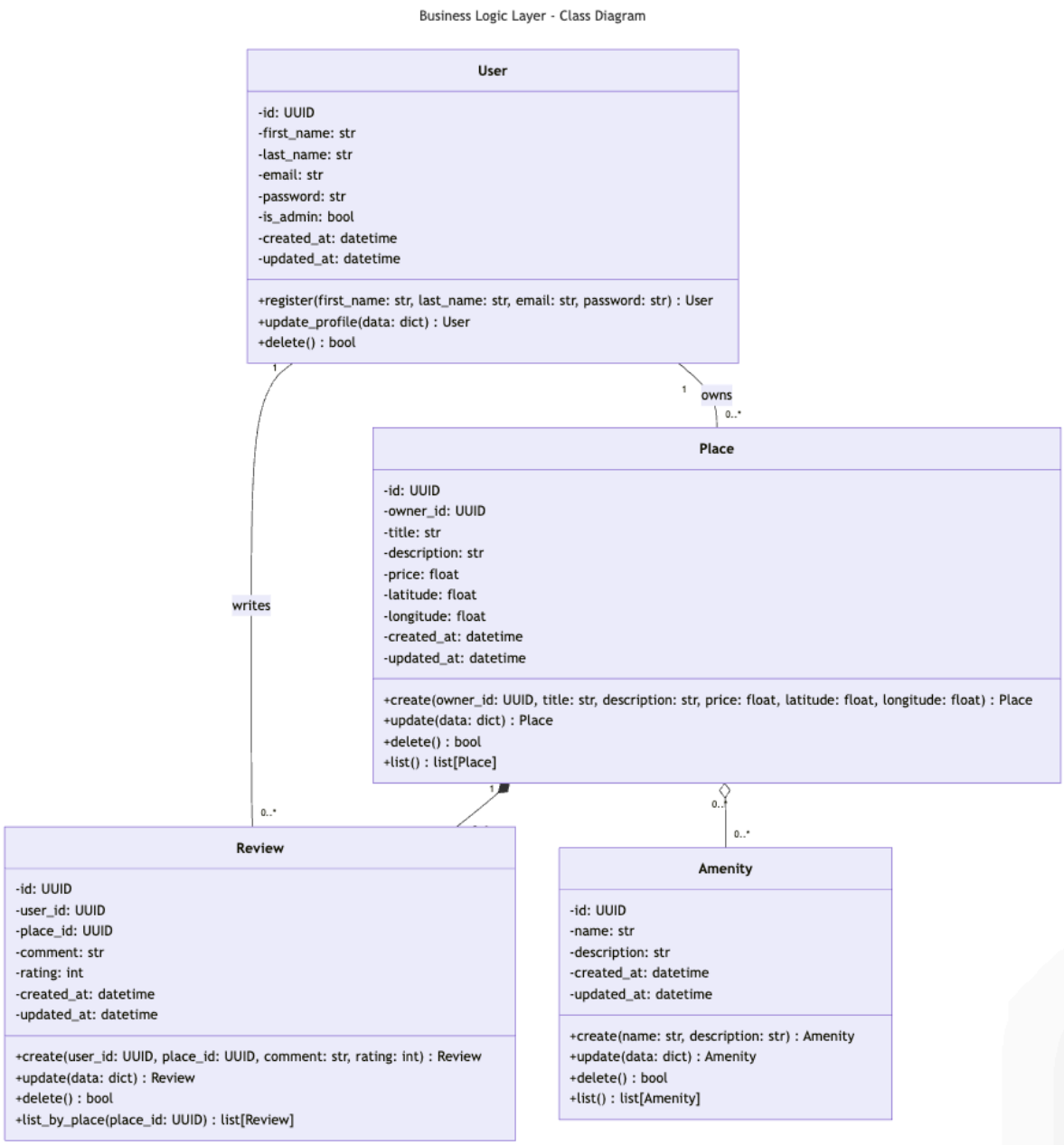
3.1 Objectif du diagramme de classes

Ce diagramme permet de :

1. Définir les entités
2. Spécifier leurs attributs
3. Illustrer les relations

4. Montrer les cardinalités
5. Vérifier le respect des règles métier

DIAGRAMME DE CLASSES UML



3.2 Description des entités

User

Représente un utilisateur de la plateforme.

Un utilisateur peut :

- Posséder plusieurs logements
- Écrire plusieurs avis

Place

Représente un logement publié sur la plateforme. Un logement :

- Appartient à un seul utilisateur
- Peut recevoir plusieurs avis
- Peut avoir plusieurs équipements

Review

Représente un avis laissé par un utilisateur.

Contraintes :

- La note est comprise entre 1 et 5
- Un avis est lié à un seul utilisateur
- Un avis concerne un seul logement

Amenity

Représente un équipement.

Un équipement :

- Peut être associé à plusieurs logements
- Existe indépendamment des logements

Il s'agit d'une **relation many-to-many** entre Place et Amenity.

3.3 Relations et cardinalités

Voici un récapitulatif des relations entre les entités :

- User 1 → 0..* Place
- User 1 → 0..* Review
- Place 1 → 0..* Review
- Place 0..* ↔ 0..* Amenity

Ces relations garantissent la **cohérence du modèle métier**.

Détails des relations :

Relation User ↔ Place (1 → 0..*)

Un utilisateur peut posséder plusieurs logements. Un logement appartient à un seul utilisateur. La cardinalité **1** du côté User signifie qu'un Place doit **obligatoirement avoir un propriétaire**.

Relation User ↔ Review (1 → 0..*)

Un utilisateur peut écrire plusieurs avis. Chaque avis appartient à un seul utilisateur.

Relation Place ↔ Review (1 → 0..*)

Un logement peut avoir plusieurs avis. Un avis est lié à un seul logement.

Relation Place ↔ Amenity (0..* ↔ 0..*)

Relation **many-to-many**. Un logement peut avoir plusieurs équipements. Un équipement peut être partagé entre plusieurs logements. Ce n'est pas une composition car un Amenity peut exister indépendamment d'un Place.

4. Diagrammes de Séquence – Flux des Appels API

4.1 Objectif des diagrammes de séquence

Les diagrammes de séquence illustrent :

- Le flux des données
- L'ordre des interactions
- La séparation des responsabilités
- La communication entre couches

Chaque diagramme montre le passage par :

Client → API → Facade → Repository → Database

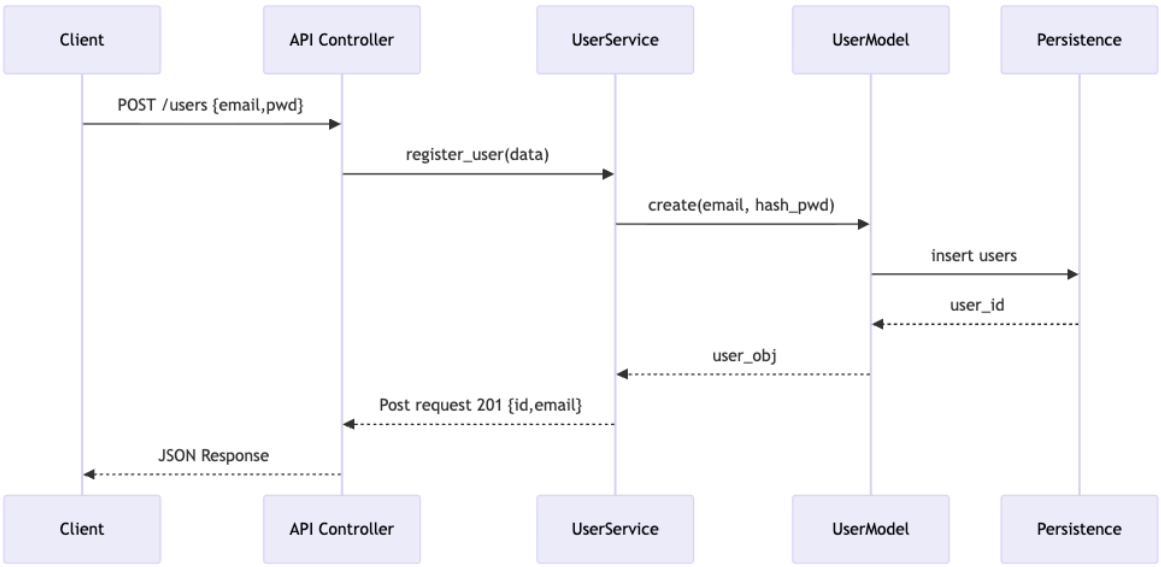
Les diagrammes de séquence montrent le **flux d'exécution** entre les composants. Chaque flèche représente :

- Un appel de méthode
- Une interaction entre objets
- Une dépendance temporaire

Les **flèches pointillées** représentent les retours.

4.2 Inscription d'un utilisateur

DIAGRAMME DE SÉQUENCE – USER REGISTRATION



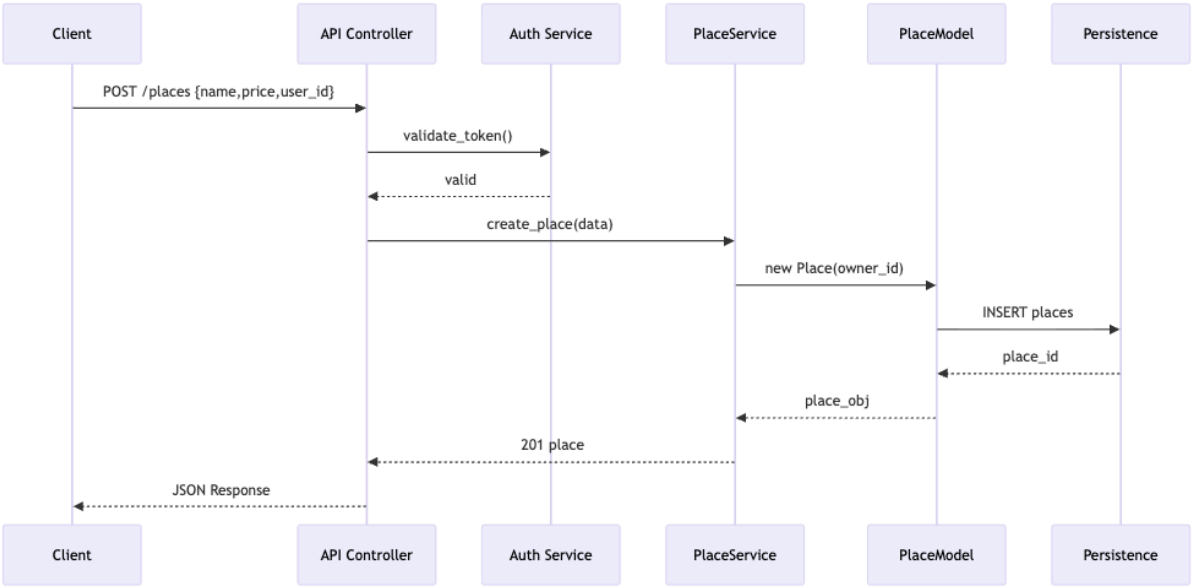
Explication :

Client → API → Facade → UserRepository → Base de données

- L'API valide les données
- La façade orchestre la création
- Le repository persiste l'objet

4.3 Création d'un logement

DIAGRAMME DE SÉQUENCE – CREATE PLACE



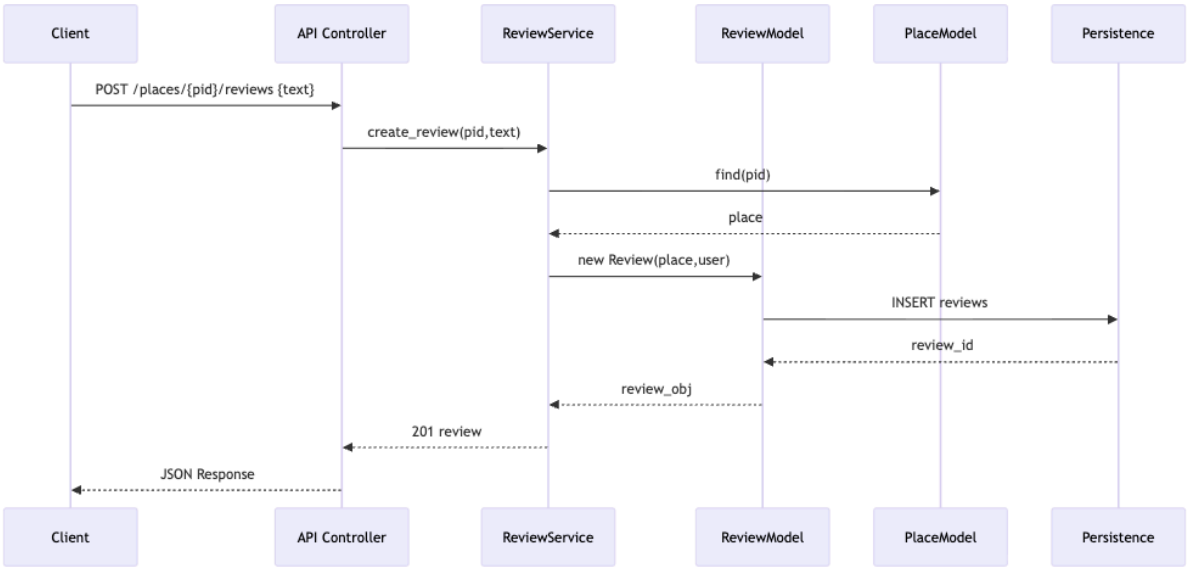
Explication :

Client → API → Facade → PlaceRepository

La façade vérifie que l'utilisateur existe avant de créer le logement.

4.4 Soumission d'un avis

DIAGRAMME DE SÉQUENCE – CREATE REVIEW



Explication :

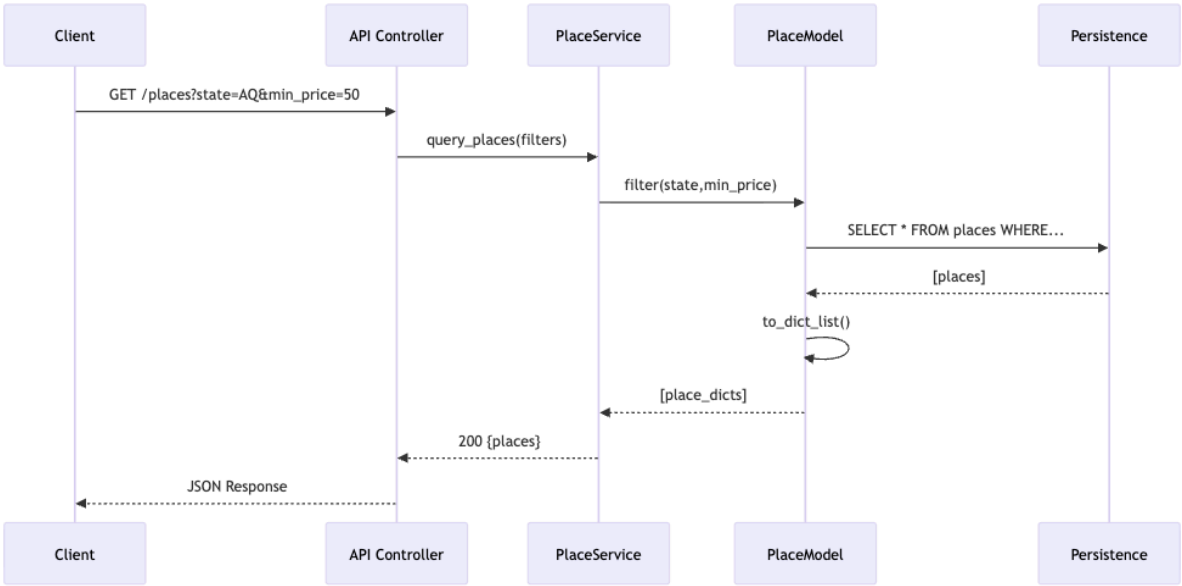
La façade vérifie :

- Existence du User
- Existence du Place
- Absence d'avis déjà existant

Ensuite elle appelle le repository pour persister.

4.5 Récupération de la liste des logements

DIAGRAMME DE SÉQUENCE – GET PLACES



Explication :

Client → API → Facade → Repository → DB

Flux simple de récupération.

5. Validation et intégrité des données

Les validations sont effectuées à **deux niveaux** pour garantir la fiabilité du système:

5.1 Validation côté API

La couche Présentation effectue les validations suivantes :

- Types de données
- Champs obligatoires
- Format des entrées

5.2 Validation métier

La couche BusinessLogic effectue les validations suivantes :

- Contraintes fonctionnelles
- Cohérence des relations
- Respect des règles métier

Cette **double validation** garantit la fiabilité du système.

6. Conclusion

Cette documentation fournit une vue complète de l'architecture et de la conception de l'application **HBnB Evolution**.

Elle inclut :

- L'architecture en couches
- Le diagramme de package
- Le diagramme de classes
- Les diagrammes de séquence
- L'explication des choix de conception
- Les règles de validation

Ce document constitue un **blueprint structuré et exploitable** pour les phases d'implémentation futures.

L'architecture de HBnB repose sur :

- Une séparation claire des responsabilités
- Une centralisation via la façade
- Une gestion propre des dépendances

Les flèches des diagrammes représentent des **dépendances logiques précises** et un flux de contrôle cohérent.

Cette architecture permet :

- Maintenabilité
- Testabilité
- Évolutivité